GOVERNANCE CONSTITUTION · COMMERCIAL EDITION

法案系统
建设说明书

轻结构 · 强执法 · 单一权威 · 失败关闭
双仓治理系统的冻结母法与执行基线

DOCUMENT VERSION
FINAL-V3.3 | 轻结构强执法闭环版

FREEZE STATUS
FROZEN-FINAL-V3.3

REVISION DATE
2026-08-25

DOCUMENT CONTROL

文档控制与阅读说明

权威载体

Markdown 为唯一权威源；本 PDF 仅供商业阅读与审阅。

治理目标

以最小结构维持逐法裁决、证据分层、唯一 Owner 与失败关闭。

适用等级

READ_ONLY、CHANGE、READY、RELEASE 四级严格区分。

执行对象

Codex、AI Agent、开发者、架构师、审计者与自动化执行者。

权威声明

本 PDF 的排版、封面、目录、页眉和页脚不产生新义务。若本 PDF 与 Markdown 存在任何差异，以同名 Markdown 文件为唯一裁决依据。

先看什么	第 0—4 节：母法身份、规范用语、授权与权威顺序。
怎么建设	第 5—34 节：两份 active 法案、Owner、按需合同与唯一 registry。
怎么执行	第 35—64 节：任务等级、证据、共享检查与唯一质量报告。
怎么防漂移	第 65—85 节：根治理准入、迁移、反例、独立自检与完成定义。

NAVIGATION

目录

0. 本说明书的唯一身份

第一部分｜最高秩序与不可变公式

1. 法案系统的核心七件套

2. 每条规则必须同时回答五个问题

3. 终态与建设阶段

4. 权威顺序与两类授权

第二部分｜固定技术底座与目录

5. 实现技术由仓库决定

6. 最小固定路径

第三部分｜定稿说明书与两份 active 法案正文

7. 本说明书的定稿保护

8. active 法案正文精确为两份

9. 两份法案的固定 front matter

10. 基础法案容纳长期稳定的根治理规则

11. 功能法案容纳随业务与能力持续迭代的法条

第四部分｜lawId 与法条原子语法

12. lawId 固定格式、lawKey 与历史谱系

13. 每条 active 法条的精确 Markdown 结构

14. law-record JSON 精确字段

15. 法条正文只写一次

16. tier 违法后果与 parser 算法

第五部分｜capabilityDomain、Owner 与 Manifest

17. capabilityDomain 命名与创建

18. 物理容器 Owner 与逻辑 law Owner 分离

19. Manifest 固定路径

20. Manifest 登记 Owner、lawIds 与路径

21. 候选文件角色与 ownership

22. 施工图按复杂度建立

23. 施工图正文

24. 规格按机器合同建立

25. 法案、Manifest、施工图、规格与质量对象的分工

第六部分｜唯一 law registry 与 canonical 生成

26. 唯一 registry

27. registry 顶层 schema

28. registry law record

29. 原生身份

30. canonical JSON

31. 唯一 registry builder

32. source-integrity preflight

33. registry 构建

34. projection check

第七部分｜任务范围与交付等级

35. 任务按交付等级分级

36. CHANGE 的实际候选

37. CHANGE 的实施边界

38. 高影响变更授权

39. 法案与既有治理迁移

40. RELEASE Git 候选与跨仓来源

41. 固定阻断码

42. Codex 范围禁区

43. 交付与迁移

44. 当前差异核验

45. AGENTS.md 的边界

46. 变更前只读检查

47. 读取规则

第八部分｜质量与证据

48. Static Audit 的边界

49. 原生 Git 候选身份

50. candidate drift

51. 一套普通治理测试

52. 静态证据分母

53. 共享检查器合同

54. 逐 lawId 裁决

55. evidenceRef

56. 唯一逐法条质量报告

57. 报告写入

58. 分级质量流程

59. 三类证据不得相互替代

60. Exam 的建立与适用

61. Dynamic Proof 的时点

62. 证据适用性

63. 唯一质量入口

64. CI 与 RELEASE 来源记录

第九部分｜根治理规则与建制

65. blank bootstrap

66. 项目根治理规则

67. 禁止递归元法案工厂

68. 唯一命令面

第十部分｜既有仓库迁移与新增能力

69. 既有仓库迁移

70. 新增 capabilityDomain

71. 新增 lawId

72. lawId 修订和退役

第十一部分｜必须阻断的漂移

73. 架构漂移

74. 范围漂移

75. 证据漂移

76. 迁移漂移

77. 文档漂移

第十二部分｜测试与独立自检

78. 普通测试与治理核心反例

79. 独立自检

第十三部分｜最终验收与完成定义

80. 法案正文验收

81. registry、施工图与规格验收

82. Owner 与变更验收

83. 交付与迁移验收

84. 质量验收

85. 状态与完成定义

附录 A｜AGENTS.md 固定正文

附录 B｜最小机器合同

B.1 Manifest

B.2 按需 blueprint machine block

B.3 按需 spec

B.4 law registry

B.5 逐法条质量报告

B.6 跨仓 provider contract payload

B.7 跨仓 consumer dependency payload

> 文件版本：FINAL-V3.3 | 轻结构强执法闭环版 冻结目标版本：FROZEN-FINAL-V3.3 修订日期：2026-08-25 文件性质：治理系统源头建设规范 / Governance Bootstrap Specification 文件定位：规定法案正文、lawId、唯一 Owner、唯一 registry、任务范围、证据等级、渐进迁移和冻结发布 适用对象：Codex、AI Coding Agent、开发者、架构师、审计者、后续自动化执行者 交付形态：Markdown 为唯一权威源；PDF 仅为可选非权威阅读副本 定稿状态：本文件保持 `FINAL-V3.3` 并默认只读；只有当前任务中的授权人明确授权才允许修改，不建立自哈希或旁路冻结文件 法条标识：全部 lawId 固定使用中性前缀 `LAW-`；任何历史项目名称、旧 fence 名称或旧 lawId 前缀不得进入本文件、法案正文、generated registry、证据或报告。

0. 本说明书的唯一身份

本说明书不是业务法案，不规定任何产品功能义务，也不替代项目的基础法案、功能法案、施工图、规格、契约或代码。

本说明书只拥有一个主权：

> 定义“法案系统本身必须如何建设、如何运行、如何防止 Codex 漂移、如何阻断第二链路”。

本说明书必须让一个从未接触项目的 Codex 在读完后能够直接回答并执行：

> 法案放在哪里；
> 法条按什么结构编写；
> 什么内容属于基础法案；
> 什么内容属于功能法案；
> lawId 怎么编号和退役；
> Owner 怎么唯一绑定；
> 唯一 registry 怎么生成和验真；
> READ_ONLY、CHANGE、READY、RELEASE 如何分级；
> CHANGE 如何保护已有改动并核对实际差异；
> 哪些范围扩张和迁移机制合法；
> 哪些越界、双重主权和无事实依据的 PASS 必须禁止；
> 质量系统怎么产出一份逐法条报告；
> Exam 和 Dynamic Proof 在什么情况下必须存在；
> 什么证据足以宣布 PASS；
> 什么情况必须停止并返回 FAIL；
> 如何防止本文件被 Codex 未经授权人许可修改。

本说明书不是可供 Codex“参考发挥”的原则集合。所有出现“固定”“必须”“只能”“不得”“精确”的条款均约束其明确点名的外部合同；Codex 不得改写这些法律效果。第 5 节、第 6 节第 9 项和第 68 节明确交给仓库的内部实现不因此被冻结。

第一部分 | 最高秩序与不可变公式

1. 法案系统的核心七件套

法案系统的核心固定由以下七类对象组成：

1. 定稿的《法案系统建设说明书》；
2. 一份《基础法案.md》；
3. 一份《功能法案.md》；
4. 由唯一 compiler 生成的 law registry；
5. 每个 capabilityDomain 唯一的 Owner Manifest；
6. 一套普通治理测试；
7. 对当前候选逐 lawId 产生裁决的一份质量报告。

唯一公式固定为：

```
法案系统
=
权威正文
+ 明确边界
+ 唯一 Owner
+ 完整索引
+ 任务范围控制
+ 可复放证据
+ 失败关闭执法
```

执法闭环固定为：

```
active lawId
-> 共享检查器按 lawId 返回独立结果
-> 全部 required evidenceModes 精确覆盖
-> 非空可复放证据
-> 一份报告逐 lawId 裁决且分母无缺失、无重复、无额外项
```

文件、检查器、测试构造器和运行过程可以共享；lawId 的最终裁决不得合并或遗漏。轻量文件结构不得降低上述执法闭环。

以下组合均不构成法案系统：

> 只有文档，没有机器执法；
> 只有测试，没有 `active lawId`；
> 只有索引，没有权威正文；
> 只有 `Owner` 名称，没有路径主权；
> 只有正向目标，没有禁止面；
> 只有 `PASS` 汇总，没有逐法条报告；
> 只有 `Codex` 提示词，没有 `active` 法案；
> 只有目录或无实质内容的占位对象，没有真实检查；
> 只有静态检查，却宣称真实运行已经成立。

2. 每条规则必须同时回答五个问题

每条 active lawId 必须精确回答：

> 要什么；
> 不要什么；
> 谁负责；
> 怎么证明；
> 违反后怎么办。

任何不能写出明确“不要什么”的要求，不得进入 active 法案。它只能继续讨论，不能取得治理主权。

2.1 边界闭合的唯一判定

“说清楚”固定等于同时写清正边界和负边界，不能只写目标，也不能只写禁令：

治理对象	要：必须做	不要：绝对禁区
法案	`summary`、唯一 Owner、required evidence、违法后果	`forbidden[]`、不得被下级覆盖
施工图	按需记录复杂拓扑、合同、状态和恢复	无实质内容的占位对象、重复法案、第二 Owner/第二真相
规格	按需记录代码之外的字段、状态、错误和 contract identity	空模板、重复法案、隐式状态
registry	忠实投影 active law、Owner、source 和按需合同	新增规范、人工维护、PASS/READY、第二索引
质检	正确证据类型、完整分母、一份逐法条报告、失败关闭	证据类型相互替代、漏项放行、warning/partial/fake PASS
任务控制	READ_ONLY 零写入、CHANGE 使用实际差异、READY/RELEASE 使用原生 Git commit	未声明写入、候选漂移、交付等级错误认定和长期双重真相

边界判定规则固定：

1. “没有写不要”不等于默认禁止；它表示边界未闭合，必须 FAIL。
2. “没有写要”不等于执行者可以自行补全；它表示生产义务未定义，必须 FAIL。
3. 合法可选项必须进入精确枚举并同时写明选择者、前置条件和失败语义；不得把“可选”当作自由发挥区。
4. 同一对象在两个位置出现不同的必做、禁区、状态、字段、错误码或输出位置时，不得择一实现；必须先消除规范冲突。

2.2 规范性用语

本说明书统一使用以下规范性用语：`必须` 表示无裁量的积极义务；`不得` 或 `禁止` 表示绝对禁止；`只能` 表示排他性路径或取值；`可以` 表示满足明示条件后的许可，不构成义务；`认定为` 表示质量或交付状态的正式裁决。正文中的说明、示例和原因不得产生未写入上述规范用语的新义务。

3. 终态与建设阶段

active 法案描述需要长期成立的生产规则；施工可以分步进行。

阶段、MVP、preview、阶段性 feature flag 和迁移窗口可以合法存在，但必须标明：

> 当前完成范围；
> 未完成范围；
> 交付等级；
> 适用人群或流量；
> 进入下一等级的条件；
> 临时机制存在时的删除日期。

阶段成果不得认定为完整生产终态；未实现或未证明的能力不得记为 PASS。

4. 权威顺序与两类授权

本说明书所称“授权人”，是指在当前任务中有权确定任务范围并批准规范修改的用户；“执行者”，是指实施或审查任务的 Codex、AI Agent 或人员。授权分为任务范围授权与规范内容授权，两者不得相互替代。

4.1 本次任务范围主权

授权人的本次任务原始请求只决定：

> 本次目标；
> 允许触碰的对象；
> 是否允许 `law-amendment`；
> 是否允许 `source-spec-amendment`；
> 是否为空白建制或既有治理迁移；

> 本次等级是 READ_ONLY、CHANGE、READY 还是 RELEASE；
>
> 是否涉及跨仓版本化合同。

任务原文不得直接降低现役法案、改变 Owner、跳过证据或授权第二链路。若普通任务原文与 active law 冲突，固定结果为 `USER_REQUEST_CONFLICTS_WITH_ACTIVE_LAW`。

4.2 规范内容主权

项目规范顺序固定为：

```
定稿的《法案系统建设说明书》
-> 《基础法案.md》
-> 《功能法案.md》
-> 按需系统或 capabilityDomain 施工图
-> 按需规格与机器契约
-> Owner Manifest
-> 唯一 law registry
-> 实现代码（由当前候选的原生 Git commit 标识）
-> 当前候选证据
-> 历史记录与讨论材料
```

解释规则固定为：

1. 授权人明确批准“修订法案”时，写权限仅及于被点名法条和完成该修订不可分割的直接关联对象；新内容仍必须符合本说明书。
2. 下级对象与上级冲突时，修改或删除下级对象；迁移期兼容机制只有满足第 43.2 节才合法。
3. 当前代码永远不能作为降低法案义务的依据。
4. generated artifact 只投影，报告只证明，消费端副本只缓存；三者均不得取得源头主权。
5. 历史报告、旧提示词、工单、会议纪要和 Git 历史只用于追溯，不能取得当前主权。

第二部分 | 固定技术底座与目录

5. 实现技术由仓库决定

除第 68 节固定的三个公共治理命令身份外，本说明书不指定编程语言、运行时、依赖库、包管理器、测试框架或内部命令。每个仓库沿用与自身业务相容的技术栈，并只保留一个现役治理实现和一个 canonical quality 入口。

工具版本、依赖版本和锁文件由仓库自己的工程配置管理。更换实现技术不得建立第二治理主链，也不得改变本说明书规定的权威、Owner、范围、证据和交付边界。

`projectChecks` 由仓库按实际构建、类型、测试和部署需要定义，不取得 law verdict。CHANGE 只运行与本次差异相关的检查；READY 和 RELEASE 再运行各自要求的完整检查。

6. 最小固定路径

基础路径：

```
AGENTS.md

.github/instructions/agent-rules/
  基础法案.md
  功能法案.md

docs/governance/
  法案系统建设说明书-冻结定稿版.md

scripts/audit/
  domains/<capabilityDomain>/manifest.json
  generated/law-registry.generated.json
  reports/law-quality.report.json

tests/governance/**
```

按需路径：

```
docs/blueprints/**
docs/specs/**
```

```
scripts/exam/**
scripts/dynamic-proof/**
```

规则固定：

1. READ_ONLY 不创建任何路径。
2. `scripts/audit/reports/**` 必须 gitignore，不得提交。
3. `scripts/audit/generated/**` 精确只有 `law-registry.generated.json`，只能由第 68 节固定的 registry 构建命令写入。
4. CI 报告写入仓库外 ephemeral path。
5. Manifest 只登记 domain、lawIds 和路径主权；不登记逐法执行器身份。
6. Exam 与 Dynamic Proof 只在 active law 声明对应 `evidenceModes` 时建立；blueprint 与 spec 只在第 22、24 节条件成立时建立。
7. 不得以 v2/new/legacy/manual/compat/shadow 后缀建立长期同职路径；第 43.2 节迁移路径必须在删除日期前退出。
8. PDF 可以从当前 Markdown 生成阅读副本；PDF 不取得权威主权，也不参加质量裁决。
9. parser、compiler、quality 和测试的内部目录、文件名及实现技术由仓库决定，不属于本说明书冻结内容。

第三部分 | 定稿说明书与两份 active 法案正文

7. 本说明书的定稿保护

本说明书固定保持 `FINAL-V3.3`。保护规则只有四条：

1. READ_ONLY、普通 CHANGE、READY 或 RELEASE 均不得自行修改本说明书。
2. 只有当前任务中的授权人明确批准 `source-spec-amendment`，才允许修改本说明书；授权范围之外的改动必须停止。
3. Git 历史和当前差异负责展示字节变化，不再建立旁路冻结清单、自哈希、派生物哈希或第二冻结状态。
4. PDF 只是从当前 Markdown 生成的阅读副本；Markdown 始终是唯一权威源，PDF 不进入 registry、报告或发布条件。

授权人明确批准修改时，可以在不改变 `FINAL-V3.3` 版本字符串的前提下对同一版本纠错或删减；不得为此建立兼容 schema、双份说明书或迁移桥。

8. active 法案正文精确为两份

人工维护并拥有 **active law corpus** 主权的文件精确为：

```
.github/instructions/agent-rules/基础法案.md
.github/instructions/agent-rules/功能法案.md
```

只有这两份文件中的 `law-record` fence 会被 active law parser 读取并进入 registry。

8.1 本说明书与 active law 的边界

本说明书直接规定法案系统本身的上位结构，不要求把 parser、compiler、registry、quality 或报告格式再次抄成 active 元法案。第 66 节只规定项目根治理法条的准入与退出条件；具体 active lawId、lawKey 和数量只由当前两份 active law source 决定。

本说明书不保存 `law-template` fence，不作为运行时 active law source，不要求一法一 spec 文件、一法一 blueprint 文件、一法一 predicate 文件或一法一反例文件。第 20、51 至 56 节规定的逐法最终裁决不因检查器共享而免除。

第 13 节只允许一个 `law-record-example` fence；它是结构示意，不进入 registry。

允许承接或投影法案、但不得创造新义务的对象只有：

```
受本说明书约束的 AGENTS.md;
按需 blueprint 与 spec;
```

```
Manifest;
唯一 law registry;
一份逐 lawId 质量报告。
```

8.2 两份源头的闭环

防止第三法案不依赖全仓关键词扫描。active-law parser 只读取第 8 节固定的两份 active law；唯一 registry builder 只把这两份法案写入唯一 registry。其它 instruction、README、AGENTS、施工图、规格、注释、任务上下文和 generated artifact 均不能成为 active law source，也不得覆盖两份 active law。parser、builder 的内部文件名、实现语言和模块拆分不属于冻结内容。

本说明书的 `law-record-example` 只用于定义 parser 合同，不进入 active law corpus。质量报告只记录当前 Git commit、逐法结果和必要的跨仓参与者，不取得法案主权。第 46 节规定变更前的只读检查，不通过旁路文件冻结 AGENTS 字节。

9. 两份法案的固定 front matter

《基础法案.md》开头精确使用：

```
---
documentType: base-law
schemaVersion: 2
status: active
authorityLevel: 100
containerOwner: law-system
lawIdPrefix: LAW
---
```

《功能法案.md》开头精确使用：

```
---
documentType: functional-law
schemaVersion: 2
status: active
authorityLevel: 90
containerOwner: law-system
lawIdPrefix: LAW
---
```

任何字段缺失、额外字段、值不一致、重复 key、重复 front matter 或第二元数据区均必须 FAIL。

本节 `containerOwner: law-system` 是两份法案文件的保留容器角色，不是必备 capabilityDomain、active lawId 或 Manifest；不得据此要求存在固定 `law-system` 法条。

10. 基础法案容纳长期稳定的根治理规则

基础法案的物理结构：

```
# 基础法案

## capabilityDomain: <stable-governance-domain>

### [LAW-<STABLE-GOVERNANCE-DOMAIN>-001-V1] ...
```

固定边界：

1. 基础法案只容纳不随普通业务迭代频繁变化、跨版本长期适用的治理 domain。
2. 项目根治理法条只有满足第 66 节准入条件时才进入基础法案；本说明书不预设必备 lawKey、编号或数量。
3. 本说明书已经直接规定的 parser、compiler、registry、quality、报告和测试机制不得仅为递归证明这些机制本身而复制成 active law。
4. parser 对基础法案与功能法案使用相同算法。
5. 新增、合并、修订或退役长期稳定治理规则必须按第 66、71、72 节处理，并取得授权人的明确批准。

基础法案不得包含产品流程、业务字段、页面、API、表、provider、业务状态机或单次任务范围。本说明书直接治理 parser、compiler、registry、quality 和报告合同；基础法案不得重复这些规则。代码纯洁度等项目级长期风险只有满足第 66 节准入条件时才可立法，不得把具体实现步骤、测试用例或报告格式写成法条。

11. 功能法案容纳随业务与能力持续迭代的法条

功能法案按 capabilityDomain 组织：

```
# 功能法案

## capabilityDomain: <domain>

### [LAW-<DOMAIN>-001-V1] <标题>
```

固定规则：

1. 业务法条及随能力变化的治理法条只属于功能法案；长期稳定治理法条只有满足第 66 节准入条件时才属于基础法案。
2. 业务 domain 与其它随能力变化的治理 domain 按 capabilityDomain 字典序排列。

3. 一个区块只容纳同名 domain 的 lawId。
4. 区块内按 lawKey 编号与版本升序排列。
5. 物理文件完整性 Owner 是 `law-system`；每个 block 的逻辑 Owner 是 metadata.capabilityDomain。
6. 修改一个 domain 不得格式化或整理其它区块。
7. parser、registry、scope、quality 的实现细节由普通测试约束，不得递归立法为固定法条工厂。

第四部分｜lawId 与法条原子语法

12. lawId 固定格式、lawKey 与历史谱系

lawId 正则固定为：

```
^LAW-[A-Z0-9]+(?:-[A-Z0-9]+)*-[0-9]{3}-V[1-9][0-9]*$
```

映射规则：

```
capabilityDomain = lower-kebab-case
lawId domain segment = capabilityDomain 转大写
lawKey = 删除末尾 -V<major> 的 lawId
```

示例：

```
capabilityDomain: account-person-workspace
lawKey: LAW-ACCOUNT-PERSON-WORKSPACE-001
lawId: LAW-ACCOUNT-PERSON-WORKSPACE-001-V1
```

编号与版本规则：

1. 每个 domain 从 `001` 单调递增。
2. V1 的 `supersedesLawId` 必须为 `null`。
3. Vn（n>1）必须精确 supersede 同 lawKey 的 V(n-1)，不得跳号。
4. requirement、forbidden、appliesTo、Owner、evidenceModes、failure 条件或规范语义任一变化，major 必须加一。
5. 同一 lawKey 只能有一个 active major；law-amendment 必须原子替换当前 major。
6. 第 43.2 节兼容窗口只适用于 spec 与实现，不允许旧、新 law major 同时进入 active law corpus；历史只在 Git 或 HISTORY_ONLY 路径。
7. 无关语义必须申请新编号，不得复用旧 lawKey。

版本关系只按当前 active law corpus 和 `supersedesLawId` 裁决。前代退役后不要求继续存在于 active corpus；parser 只验证 `supersedesLawId` 与当前 lawId 具有相同 lawKey，且 major 精确为当前 major 减一，不为此建立历史 registry。Git 历史只用于人工追溯，不进入普通 quality，也不得成为当前主权或 PASS 前提。

13. 每条 active 法条的精确 Markdown 结构

下面使用 `law-record-example` 只为避免本说明书自身被 active parser 误识别。active 两份法案中的实际 fence label 必须是 `law-record`：

````
### [LAW-<DOMAIN>-001-V1] <中文标题>

```law-record
{ ...canonical law JSON... }
```
````

active-law parser 必须把上述内层 `law-record` 视为被外层示例 fence 包裹的数据，不得把它计入 active block；active parser 只接受两份法案中未嵌套、未缩进、行内容精确等于三个反引号加 `law-record` 的 fence。

法条 section 不得增加背景故事、当前实现盘点、开发日志、TODO、方案比较或未进入 metadata 的隐藏要求。law-record 只规定有期限迁移义务；具体期限、日期和删除验收进入关联 migration spec。实现落位进入按需 blueprint，机器合同进入按需 spec。

14. law-record JSON 精确字段

每个 block 只允许：

```
{
  "schemaVersion": 2,
  "lawId": "LAW-<DOMAIN>-001-V1",
  "supersedesLawId": null,
  "status": "active",
  "tier": "P0",
  "capabilityDomain": "<lower-kebab-case>",
  "enforcementMode": "static",
  "evidenceModes": ["static.ast"],
  "summary": "必须……",
  "appliesTo": ["src/<domain>/**"],
  "excludes": [],
  "forbidden": ["不得……"],
  "dependsOnLawIds": []
}
```

字段约束：

14.1 schemaVersion、status、supersedesLawId

```
schemaVersion 固定整数 2;
status 固定 active;
V1 supersedesLawId=null;
Vn 精确指向同 lawKey 的 V(n-1);前代已退役时无需保留其 active block,亦不得为验证版本关系建立历史 registry。
```

14.2 tier

只允许 `P0`、`P1`、`P2`；全部都是失败关闭，不存在 warning law。

14.3 enforcementMode 与 evidenceModes

`enforcementMode` 只允许 `static`、`exam`、`dynamic`、`composite`。

```
只有 static.* -> static
只有 exam.* -> exam
只有 dynamic.* -> dynamic
存在两类或三类 -> composite
```

`evidenceModes` 只允许：

```
static.filesystem
static.text
static.ast
static.types
static.contract
static.dependency
static.config
static.json
static.yaml
static.sql
static.migration
exam.golden
exam.property
exam.state-machine
dynamic.browser
dynamic.service
dynamic.database
dynamic.storage
dynamic.provider
dynamic.authorization
```

```
dynamic.recovery
dynamic.performance
```

数组非空、无重复、按上表排序。

每个 law-record 只治理一个可以独立修订或退役的规范主题，并只产生一个最终裁决。具有相同 Owner、`appliesTo/excludes`、evidenceModes 和 tier，且必须共同成立才能关闭同一风险的正负条件可以写入同一 law-record；其中任一条件可以独立修订、退役或改变证据等级时，必须拆分 lawId。不得为了句子、字段、parser 分支或测试用例机械拆法，也不得建立平行 clause registry。执法分母的唯一 key 是 `lawId + evidenceMode`。

evidenceModes 的最低选择规则固定为：只涉及仓库静态结构的事实使用 `static.*`；涉及可重复执行的确定性行为至少使用 `exam.*`；涉及真实浏览器、服务、数据库、存储、provider、授权、恢复或性能事实至少使用对应 `dynamic.*`；同时涉及多类事实时必须声明全部类别并使用 `composite`。不得通过少报 evidenceMode 降低法条实际要求的证明等级。

14.4 summary

```
以“必须”或“只能”开头；
只描述第 14.3 节的一个治理风险；
非空且只包含完成该规范主题所需的文字；
不得出现建议、尽量、可能、一般、视情况或原则上；
不得写开发日志、文件清单或单次任务范围；有期限迁移的机器字段进入第 69 节 migration contract，不塞入法条 summary。
```

14.5 appliesTo 与 excludes

```
仓库相对 POSIX glob；有效 UTF-8 NFC；不得绝对路径、..、反斜杠、负向 glob、brace、extglob、字符组或问号；
只允许精确文件、目录 /**、目录 /**/*.<literal-extension>；
数组字典序、无重复、无包含冗余；
appliesTo 非空；
只有基础法案中满足第 66 节准入条件且确需观察全仓的根治理法条允许在 `appliesTo` 使用根级 `**/*`；
excludes 只能排除 appliesTo 内明确子面，不得排除生产主链逃避执法。
```

14.6 forbidden

数组非空；每项以“不得”或“禁止”开头；一项一个违法条件；相同违法条件不得换词重复。不得以模糊阶段词豁免现役义务。有期限迁移的法条写明不得无期限并由关联 migration spec 提供第 43.2、69 节机器字段，不视为豁免。

14.7 dependsOnLawIds

只能引用当前 active lawId；按 lawId 排序且无重复；不得自引；依赖图无环；跨域依赖必须进入 registry，并在存在施工图时同时反映于相关施工图；不得分散 Owner。

15. 法条正文只写一次

`law-record` 是法条唯一正文。标题只提供阅读导航，不重复 summary、forbidden、evidenceModes 或违法后果。parser 验证标题 lawId、domain、版本关系和 JSON 字段，不维护第二份逐字镜像。

16. tier 违法后果与 parser 算法

tier 的违法后果固定为：

> P0：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P0 finding；对应质量入口必须非零退出，并阻断合并、发布和用户交付。
>
> P1：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P1 finding；对应质量入口必须非零退出，并阻断合并和发布。
>
> P2：任一要求不成立、任一禁止条件出现或任一必需证据缺失，即产生 P2 finding；对应质量入口必须非零退出，并阻断合并。

16.1 law-parser 的精确解析算法

> 1. 读取原始字节；拒绝 BOM、非 UTF-8、非 NFC 路径和 CRLF；
> 2. 验证 front matter 从首字节开始，以精确 `---\n` 打开和关闭；
> 3. 唯一 yaml parser 解析，禁止重复 key；
> 4. 逐行维护 Markdown fence 状态；
> 5. 只有精确行 ```` ```law-record ```` 打开法条 block；只有精确行 ```` ``` ```` 关闭；
> 6. 禁止未关闭、嵌套、缩进 fence 和 block 内注释；
> 7. block 必须是单一 JSON object，禁止未知字段；
> 8. 找到最近 level-3 law heading，一个 heading 精确一个 block；
> 9. 找到最近 level-2 capabilityDomain heading；基础法案和功能法案算法相同；
> 10. 验证标题 lawId、domain、major 与 supersedes；
> 11. 记录原始 lineStart/lineEnd；
> 12. EOF 后拒绝孤立 law heading 或 law-record；
> 13. 任一解析错误只产生结构冲突，不生成部分 registry。

16.2 其它 parser

Manifest、blueprint、spec 的 parser 必须拒绝重复 key、未知字段、空字符串、非法路径、未替换占位符和默认补值。其内部文件名、实现语言和模块拆分由仓库决定。

第五部分｜capabilityDomain、Owner 与 Manifest

17. capabilityDomain 命名与创建

capabilityDomain 正则固定为：

```
^[a-z0-9]+(?:-[a-z0-9]+)*$
```

每个 capabilityDomain 必须代表一个能够独立回答下列问题的完整责任域：

它拥有哪一种事实或能力；
它创建什么；
它读取什么；
它修改什么；
它裁决什么；
它明确不得接管什么。

不得创建以下 domain：

```
common、shared、utils、misc、core2、platform2、general、helpers、temporary、legacy、compat。
```

新 capabilityDomain 只有在授权人明确批准新增能力域，并且现有 domain 无法合法承接时才允许创建。执行者不得仅为整理代码、改善形式或减少文件行数自行新增 domain。

18. 物理容器 Owner 与逻辑 law Owner 分离

固定关系：

```
基础法案.md / 功能法案.md 的文件完整性 Owner = 第 9 节保留容器角色 law-system
每个 law-record block 的逻辑 Owner = metadata.capabilityDomain
```

因此：

一个物理文件可以容纳多个逻辑 Owner；
一个 lawId 只能有一个逻辑 Owner；

> Owner 不得通过拥有法案文件路径接管其它 lawId；
>
> Audit 必须按 lawId block 定位主权，不得按 Markdown 文件整体绑定。

19. Manifest 固定路径

每个 active capabilityDomain 精确存在：

```
scripts/audit/domains/<capabilityDomain>/manifest.json
```

目录名与 `manifest.capabilityDomain` 必须相同。Manifest 只登记 domain 的 active lawIds 与路径主权，不得成为第二法案、执行器清单、独立测试计划或报告配置。

普通治理测试和检查器可以跨 domain 组织；其文件名和模块拆分由仓库决定。quality 必须把共享执行结果拆成逐 lawId 结果。

20. Manifest 登记 Owner、lawIds 与路径

每个 capabilityDomain 的 Manifest 顶层只允许：

```
{
  "schemaVersion": 5,
  "capabilityDomain": "<lower-kebab-case>",
  "status": "active",
  "lawIds": [],
  "ownedPaths": [],
  "observedPaths": []
}
```

规则固定：

1. `lawIds` 精确等于该 domain 当前 active lawId。
2. `ownedPaths` 表示唯一 primary 写主权，只允许精确文件或目录 `/**`；`observedPaths` 只读，除前述形式外还允许根级 `**/*`。根级 `**/*` 不得进入 `ownedPaths`。Owner 重叠只按 `ownedPaths` 裁决，必须 FAIL；`observedPaths` 不取得写主权，允许多个 domain 观察同一路径。
3. `lawIds` 按 lawId，路径数组按 UTF-8 bytes 升序；全部无重复。
4. Manifest 不得写检查器、测试、法条正文、PASS 条件、豁免、report sink 或新义务。
5. active law、Manifest、registry 与质量报告的 lawId 集合必须精确相等；检查器共享不得改变该分母。

21. 候选文件角色与 ownership

候选路径只分为：

```
GOVERNANCE_AUTHORITY;
GENERATED_REGISTRY;
QUALITY_REPORT;
TEST_ONLY;
HISTORY_ONLY;
PRODUCTION。
```

规则固定：

1. 本说明书、AGENTS、两份 active law、Manifest 及按需 blueprint/spec 为 GOVERNANCE_AUTHORITY。
2. 第 26 节唯一 registry 为 GENERATED_REGISTRY；第 57 节报告为 QUALITY_REPORT。仓库外临时证据不属于候选路径。
3. `tests/**` 及未被生产消费的 fixture/example 为 TEST_ONLY；明确 `docs/history/**` 为 HISTORY_ONLY；其余候选路径默认为 PRODUCTION。
4. PRESENT 的 PRODUCTION 文件必须命中一个且仅一个 Manifest `ownedPaths`。
5. 非生产路径被生产 import、发布包、配置或状态写入消费时，按 PRODUCTION 裁决。
6. DELETED 不读取工作树字节；删除是否合法按当前 Git 基准树中的原 Owner 与当前任务授权验证。
7. rename 视为旧 path DELETE 与新 path PRESENT。
8. symlink 不得越出仓库根；submodule、unmerged index 和特殊设备文件必须阻断。

固定结果：

```
零 Owner -> UNOWNED_PRODUCTION_FILE;
多 Owner -> MULTIPLE_PRIMARY_OWNERS;
非生产角色被生产消费 -> ROLE_ESCALATION_REQUIRED;
未授权删除 -> UNAUTHORIZED_PRODUCTION_DELETE。
```

22. 施工图按复杂度建立

`SYSTEM-BLUEPRINT.md` 与 capabilityDomain blueprint 不再是每个仓、每个 domain、每条法案的强制配件。

出现以下任一情形时，才必须建立机器可读施工图：

```
存在无法由单份 spec 与代码引用直接确定的多 domain 依赖拓扑;
多份跨域或跨仓合同之间存在必须集中裁决的调用顺序或禁止方向;
```

> 多状态恢复流程；
>
> 无法从代码与 `spec` 直接确定的 `Owner` 或调用方向。

施工图的最小机器 block：

```
{
  "schemaVersion": 1,
  "blueprintId": "<stable-id>",
  "status": "active",
  "owner": "<capabilityDomain>",
  "lawIds": ["LAW-<DOMAIN>-001-V1"],
  "contractRefs": [],
  "dependencyEdges": [],
  "prohibitedDependencyEdges": []
}
```

每个 blueprint 的 `lawIds` 与 `contractRefs` 精确定义该 blueprint 的 law-to-contract 关系；registry 同时投影第 24 节 active specs 的显式 `lawIds`。不得依赖 Markdown 标题、行号或自然语言推断关系。

`lawIds` 按 lawId、`contractRefs` 按路径的 UTF-8 bytes 升序排列，均无重复。

edge endpoint 精确为 `{repo, domain}`。`repo` 只允许字面值 `local` 或 active cross-repo contract 中的 canonical `owner/repo`；`domain` 必须是对应仓的现役 capabilityDomain。`from` 表示依赖方或调用方，`to` 表示被依赖方或提供方；两端不得相同。

`dependencyEdges[]` item 只允许 `from`、`to`、`contractRef`。`contractRef` 只允许 null 或 top-level `contractRefs` 中的 active spec 路径；跨域或跨仓同存在时不得为 null。

`prohibitedDependencyEdges[]` item 只允许 `from`、`to`、`lawId`。parser 只验证 `lawId` 同时存在于 top-level `lawIds` 且为 active；该 law 是否禁止此方向由普通治理测试按明确合同和依赖关系裁决，不要求 parser 解释自然语言。施工图不得自行创造禁令。

两个数组分别按 `from.repo`、`from.domain`、`to.repo`、`to.domain`、最后一个字段的 UTF-8 bytes 升序排列。同一数组不得重复；相同 `from/to` 不得同时出现在允许与禁止数组；未知字段、非法 endpoint、悬空引用、逆序、重复或允许/禁止冲突必须 FAIL。

blueprint 可以是独立 JSON 文件，也可以是 Markdown 中唯一的 `blueprint-record` JSON fence；builder 只读取该机器 block，不解析正文形成隐含合同。

施工图只记录 topology、Owner、contract 和 prohibited edge；不得为每个 lawId 建立形式化 anchor。

简单 domain 可以只使用法案、Manifest、代码与测试，不得为了满足目录形式创建空 blueprint。

23. 施工图正文

施工图只写当前系统需要长期维持的结构，不设固定十五章，不要求空章节。

正文必须覆盖实际存在的：

> 终态对象；
> Owner；
> 输入输出；
> 持久化与状态；
> 调用方向；
> 错误恢复；
> 安全边界；
> 验收条件。

不存在的对象不得以占位章节、TODO 或未来规划写入。

24. 规格按机器合同建立

只有法条涉及字段、API、事件、存储、状态机、配置、安全合同或跨仓合同，且代码类型本身不足以成为唯一机器真相时，才建立 spec。

spec 必须具有：

> specId；
> version；
> Owner；
> 适用 `lawIds`；
> 精确合同内容。

spec 的最小 schema：

```
{
  "schemaVersion": 1,
  "specId": "<capabilityDomain>.<stable-id>",
  "status": "active",
  "version": 1,
  "owner": "<capabilityDomain>",
  "lawIds": ["LAW-<DOMAIN>-001-V1"],
  "specKind": "typed-contract",
  "contract": {}
}
```

`lawIds` 按 lawId 排序且无重复。

`specKind` 只允许 `typed-contract`、`api`、`event`、`storage`、`state-machine`、`config`、`security`、`migration`、`cross-repo-contract`、`cross-repo-dependency`。`specKind` 唯一决定 `contract` 使用的 schema；同一 `specKind` 和 schemaVersion 不得同时对应两套 schema，也不得由文件路径、Owner 或运行参数改选 schema。spec parser 校验公共外壳后，按 `specKind` 进入唯一合同 schema；schema 的内部实现位置和技术由仓库决定。公共外壳未知字段、缺失 Owner、空 lawIds、未知 specKind、合同不符合对应 schema 或未被 active law 引用必须 FAIL。

`version` 必须是正整数；`specId + version` 在当前 active specs 中必须唯一。`contract` 的规范内容发生变化时递增；旧、新版本同时 active 只允许发生在第 43.2 节兼容窗口内，并必须由同一 migration spec 规定各自生效范围和旧版本删除日期。spec 不得重复法条正文，不得为每条 lawId 强制建立一份文件，不得用空数组和模板字段制造形式完整。

跨仓 provider 只保存一份权威合同。consumer 只声明 `contractId`、`version` 与 `providerRepo`，不得复制 provider 合同内容或绑定 provider commit/hash。provider 合同的规范内容变化必须提升 `version`；provider 的 CHANGE、READY 和 RELEASE quality 必须以原生 Git diff 核对合同内容变化与 version 递增，不得让同一 version 在后续 commit 中改变语义。消费者可以独立完成 CHANGE 与 READY；RELEASE 才按第 40、56 节从必要 provider participant 的原生 Git commit 中查找唯一匹配的 active provider 合同并核对版本，不要求同一任务同步修改或发布全部仓库。

25. 法案、Manifest、施工图、规格与质量对象的分工

> 法案：规定必须、禁止、Owner、证据等级与违法后果；
>
> Manifest：登记 domain、lawIds 与路径主权；
>
> 施工图：按需记录复杂拓扑和调用边界；
>
> spec：按需记录代码之外的机器合同；
>
> registry：投影当前 active laws、Owner 与合同引用；
>
> 质量报告：逐 lawId 记录当前候选的裁决与证据。

任何对象不得重复取得另一对象的主权。

第六部分｜唯一 law registry 与 canonical 生成

26. 唯一 registry

唯一机器索引固定为：

```
scripts/audit/generated/law-registry.generated.json
```

不得建立平行 law index、authority census、执行器登记文件、audit-plan 文件、inspector-registry 文件或第二 generated projection。

registry 只投影当前 active law、Owner、source location、evidenceModes 和 contract refs；不拥有规范主权、执行器身份或 PASS 权。

27. registry 顶层 schema

```
{
  "schemaVersion": 5,
  "sourceSpec": {"path": "..."},
  "lawCorpus": {
    "baseLawPath": "...",
    "functionalLawPath": "..."
  },
  "laws": [],
  "domains": []
}
```

`domains[]` 每项只允许：

```
{
  "capabilityDomain": "...",
  "manifestPath": "...",
  "lawIds": [],
  "ownedPaths": [],
  "observedPaths": []
}
```

全部数组按稳定 key 排序；禁止时间、机器名、用户名、绝对路径和随机值。

28. registry law record

```
{
  "schemaVersion": 2,
  "lawId": "LAW-...",
  "status": "active",
  "supersedesLawId": null,
  "tier": "P0",
  "capabilityDomain": "...",
  "enforcementMode": "static",
  "summary": "...",
  "appliesTo": [],
  "excludes": [],
  "forbidden": [],
  "evidenceModes": [],
  "dependsOnLawIds": [],
  "source": {
    "path": "...",
    "lineStart": 1,
    "lineEnd": 10
  },
  "contractRefs": []
}
```

`contractRefs` 只收录 `lawIds` 包含该法条的 active blueprint/spec 路径字符串，按 UTF-8 bytes 升序排列；没有按需合同的法条必须为空数组。Git commit 已绑定这些仓库内文件，不再为引用重复计算文件 hash。

29. 原生身份

仓库内的说明书、法案、registry、代码、测试和合同统一由原生 Git commit 绑定，不再为每个文件、法条 section 或 registry 重复建立 hash 身份。

Git object id 使用当前仓库 object format 的小写十六进制，不固定为 40 位；同一候选中的 Git id 必须使用同一 object format。

仓库外或二进制证据不受 Git commit 保护时，可以使用平台原生 receipt ID 或 SHA-256；不得把这项例外扩展回普通仓库文件。

30. canonical JSON

对象 key 按 Unicode code point 升序递归排序；数组保持其所属 schema 规定的顺序。registry 的 `laws` 按 lawId、`domains` 按 capabilityDomain、`contractRefs` 按 path 排序；字符串数组按各自字段规则排序。缩进 2；单 LF 结尾；禁止注释、尾空格、NaN、Infinity、undefined 和时间字段。

31. 唯一 registry builder

仓库只允许一个承担 registry 生成主权的 builder，并由第 68 节固定的 `audit:build` 调用。builder 的内部文件名、实现语言和模块拆分由仓库决定。

输入只包括：

```
本说明书；
当前 AGENTS；
两份 active 法案；
全部 active Manifests；
固定按需目录中全部 status=active 的 blueprints/specs。
```

唯一输出是 `law-registry.generated.json` 的 canonical bytes。builder 必须验证 active law、Manifest、contract refs 与 lawId 分母精确对等；不读取生产候选、检查器实现、历史报告、网络、数据库、浏览器、墙钟或随机源，不输出 verdict。

32. source-integrity preflight

builder、projection check 与 READY、RELEASE quality 共用以下源头只读检查：

```
说明书路径与版本；
AGENTS 是否符合第 45 节；
两份 active law 结构；
active law source 精确为固定两份；
Manifest Owner 与路径重叠；
active law 与 Manifest 的 lawId 全分母对等；
按需 blueprint/spec 的 schema、lawIds 与 Owner 完整性；
path encoding 与 case collision。
```

`audit:build` 在 registry 不存在时也必须能够执行，只验证源头、Manifest 与按需合同后生成 registry。`audit:projection:check`、READY 与 RELEASE 才额外要求现有 registry 与同一 builder 的内存结果 exact bytes 相等，并验证 registry lawId 分母。CHANGE 只验证说明书、AGENTS 边界及本次实际触碰的治理 source；不遍历全量 registry、ownership 或未受影响合同。

失败只输出确定性 code，不写 registry 或报告。

33. registry 构建

`audit:build` 在内存生成唯一 registry，验证可重新解析后以临时文件和原子 rename 替换目标文件。失败时保留旧文件，不得留下半写状态。

`audit:build` 不写质量报告，不自动补法案、Manifest、blueprint 或 spec。

34. projection check

`audit:projection:check` 重新调用同一 builder，比对唯一 registry exact bytes，并证明检查前后候选未变化。

registry 缺失返回 `GENERATED_FILESET_INVALID`；字节陈旧或被手改返回 `GENERATED_PROJECTION_STALE`。检查不得自动调用 `audit:build` 修复。

第七部分 | 任务范围与交付等级

35. 任务按交付等级分级

任务只分为 `READ_ONLY`、`CHANGE`、`READY`、`RELEASE` 四级。不得将较低等级认定为较高等级。

分类顺序固定：授权人明确禁止写入时为 READ_ONLY；明确要求生产发布、最终发布证明或 `SYSTEM_PROVEN` 时为 RELEASE；明确要求合并、部署前就绪或 READY 验收时为 READY；其它包含仓库写入的任务为 CHANGE。执行者不得自行降级；授权人未要求更高等级时不得自行升级。

35.1 READ_ONLY

READ_ONLY 只读取权威与目标文件并回答，不写入任何文件。

只读结论不得宣称代码已经修改、候选已经验证或系统已经发布。

35.2 CHANGE

CHANGE 直接按授权人的请求实施修改。开始修改前只读查看目标路径的 Git 状态；目标已有用户改动且授权人未明确允许在其上继续编辑时，立即停止。

CHANGE 由执行者在当前任务上下文中核对实际差异与授权人的请求，并通过与差异相交的普通测试；仓库命令只能证明其实际读取和执行的差异、检查与测试，不能证明自己未读取的授权人原始请求或修改前用户状态。
CHANGE 不得据此宣称 READY、RELEASE 或 `SYSTEM_PROVEN`。

35.3 READY

READY 只接受 clean Git commit，并表示该 commit 已通过全部 projectChecks、全部 active Static、全部 active Exam、治理核心反例测试和完整逐法报告；声明的 `dynamic.*` 尚未取得。

READY 不得替代真实生产环境、外部依赖或动态运行证据。

35.4 RELEASE

RELEASE 只接受 clean Git commit，是唯一执行全部 active evidenceModes、全量逐法报告和必要跨仓参与者核验的等级。

只有 RELEASE 可以产生 `SYSTEM_PROVEN`。

36. CHANGE 的实际候选

CHANGE 以当前 Git HEAD 与工作树的实际差异作为候选，不维护预先声明的路径集合。

规则固定：

1. 实际差异必须包含 tracked 文件的新增、修改、删除，以及未跟踪文件。
2. Git HEAD 只作为本次检查时的当前比较基准，不写入任务文件。
3. 开始修改前只读检查目标路径状态；不得覆盖、删除、恢复或移动未获授权的用户改动。
4. 检查过程中实际差异变化时，本次临时结果作废并重新检查。
5. CHANGE 不生成候选回执、路径清单或其它任务 artifact。

37. CHANGE 的实施边界

实施内容仅来自：

> 授权人明确点名的路径；
>
> 完成该修改不可分割的同一合同路径；
>
> 由唯一生成器确定发生变化的派生路径。

第二、三项只有在完成授权事项确有必要时才实施；不得以整理、抽象、未来复用或附带修复为由扩大修改。

READ_ONLY 不执行本节。

38. 高影响变更授权

新增 API、CLI、事件、队列、表、列、环境变量、依赖、监听器、外连主机、浏览器权限或跨仓合同，首先必须属于第 37 节授权范围；active law 只能证明规范必要性，不能扩大任务范围。授权人的请求没有明确包含该新增事项，且完成授权事项并非不可分割地需要该新增事项时，执行者必须停止并请求批准。

唯一生成器派生物、完成同一合同不可分割的修改、active law 与当前等级直接要求的证据，以及完成旧路径退休不可分割的修改，可以随请求一并实施；其余内容不属于当前 CHANGE。

39. 法案与既有治理迁移

修改本说明书必须取得授权人对 `source-spec-amendment` 的明确批准；授权只覆盖本说明书及其非权威 PDF 阅读副本，不自动授权修改 AGENTS、active law、registry 或实现。Git 差异必须完整展示正文变化，不生成自哈希或冻结旁路文件。

39.1 法案修订

修改 active 法案必须取得授权人对 `law-amendment` 的明确批准。

修订可以在分支、工作树或隔离目录中验证；不得要求所有关联实现必须在同一任务达到 RELEASE。修订后的 active law、registry 和实现只有在各自完成时才取得对应状态，未完成项必须如实标记；law-amendment 不改变本说明书版本。

39.2 既有治理迁移

既有治理允许分步迁移。每一步必须在任务交付说明中列明当前权威、目标权威、受影响路径、验证方式和剩余工作；中间状态不得认定为 READY 或 RELEASE。存在第 43.2 节兼容机制时，另按第 69 节写入 migration spec；没有兼容机制时不创建迁移 artifact。

迁移期间允许第 43.2 节规定的有期限兼容机制。

40. RELEASE Git 候选与跨仓来源

进入 RELEASE 时必须固定：

```
Git commit；
全部 required Dynamic Proof 的真实 locator 与原生 receipt；
跨仓合同版本；
每个必要参与仓的 repo 与 Git commit。
```

上述字段直接进入第 56 节唯一质量报告，不再生成自定义候选身份、按候选建目录、第二发布证明包或第二 release receipt。任一 participant 缺失、合同版本不兼容或 commit 漂移时，RELEASE 失败。

41. 固定阻断码

任务控制的跨实现稳定阻断码至少包括：

```
USER_REQUEST_CONFLICTS_WITH_ACTIVE_LAW
LAW_AMENDMENT_REQUIRED
SOURCE_SPEC_AMENDMENT_REQUIRED
RELEASE_GIT_COMMIT_REQUIRED
READY_GIT_COMMIT_REQUIRED
RELEASE_EVIDENCE_MISSING
CANDIDATE_DRIFT
EXTERNAL_SURFACE_AUTHORITY_MISSING
```

源头、ownership 与 projection 至少使用：

```
SOURCE_PARSE_FAILED
GENERATED_FILESET_INVALID
GENERATED_PROJECTION_STALE
UNOWNED_PRODUCTION_FILE
MULTIPLE_PRIMARY_OWNERS
```

```
ROLE_ESCALATION_REQUIRED
UNAUTHORIZED_PRODUCTION_DELETE
```

逐法执法闭环至少使用：

```
LAW_EVIDENCE_MISSING
LAW_EVIDENCE_DUPLICATE
UNDECLARED_EVIDENCE_MODE
CHECK_EXECUTION_FAILED
EMPTY_EVIDENCE_REFS
LAW_DENOMINATOR_MISSING
TARGET_OUTSIDE_DOMAIN_AUTHORITY
ACTIVE_LAW_DENOMINATOR_MISMATCH
LAW_FAILED
```

以上是跨实现必须保持语义稳定的最低集合，不是禁止新增确定性错误码。仓库可以为具体失败事实增加不重名的 code，但不得为同一语义建立 alias，也不得用笼统 code 吞掉本节已经命名的失败。

READ_ONLY 只在目标或权威不可读取时停止，不要求任何任务 artifact。

42. Codex 范围禁区

未经明确授权，Codex 不得：

```
偏离授权人的请求及完成该请求不可分割的直接关联对象；
新增产品范围、公开接口、持久化结构或外部权限；
建立第二法案、第二 registry、第二 Owner、第二 compiler 或第二质量入口；
将 fixture、mock、no-op 或固定返回认定为生产能力；
将 CHANGE 或 READY 结论认定为 RELEASE；
覆盖、删除、恢复或移动用户原有改动。
```

迁移桥、用于迁移的 feature flag、dual-read、dual-write 和 shadow traffic 依第 43.2 节裁决，不作绝对禁止。长期产品 feature flag 由对应业务 law 管理，不得用于暗中保留旧真相源。

43. 交付与迁移

43.1 渐进建设

渐进建设精确使用第 3 节规则，不新增状态或豁免。

43.2 有期限兼容机制

兼容桥、用于迁移的 feature flag、dual-read、dual-write、shadow traffic 或临时 adapter 只有同时满足以下条件才合法：

> 唯一 Owner；
> 旧、新路径身份明确；
> 开始日期；
> 监控指标；
> 回滚条件；
> 最长存续期限；
> 删除日期；
> 删除验收标准。

兼容机制不得形成两个长期真相源；到期未删除必须 FAIL。

43.3 跨仓交付

跨仓通过版本化合同独立部署。合同内容变化递增 `version`；破坏性改变同时声明兼容窗口和消费者迁移期限。

一仓 CHANGE 或 READY 不得认定为另一仓已经完成；只有整体 RELEASE 声明才要求全部必要参与仓的发布结果。

44. 当前差异核验

CHANGE 以当前 Git HEAD 到工作树的实际差异为分母；READY 与 RELEASE 以 clean Git commit 为分母。

CHANGE 的范围义务分为执行者核对与机器检查，二者不得相互替代。执行者必须在当前任务上下文中核对：

> 实际差异与授权人的请求及完成该请求不可分割的直接关联对象一致；
> 未获授权的用户改动未被覆盖、删除、恢复或移动；
> 法案或本说明书修改已经取得对应授权；

由于 CHANGE 不保存授权人原始请求、修改前快照或范围事务，`quality:check CHANGE` 不得宣称已经自动证明上述三项。它只根据调用时的实际 Git 差异执行可确定的 source 结构检查、授权门和与差异相交的 project checks/普通测试；最终交付必须由执行者明确报告范围核对和用户原有改动保护情况。

READY 与 RELEASE 的 `quality:check` 必须验证：

> 当前候选是 clean Git commit；
> generated bytes 与唯一生成器对同一输入的输出完全一致；
> 全部生产路径具有唯一 Owner；

> 全部外部 surface 均由 active law 及其精确 spec/contract 授权，并具有唯一 Owner；
> 检查前后 Git commit 未变化。

任一不成立，返回第 41 节对应阻断码并非零退出。

45. AGENTS.md 的边界

AGENTS 只写权威顺序、READ_ONLY 零写入、CHANGE 的 Git 状态保护、交付等级、READY/RELEASE 使用原生 Git commit、有期限迁移、本说明书与 active law 的修改授权和失败关闭；不得复制业务法案、手工路由表或完整质量流程。

46. 变更前只读检查

READ_ONLY 直接读取，不运行治理命令。

CHANGE 开始修改前只读查看说明书、AGENTS 边界、唯一质量入口和目标路径 Git 状态。READY、RELEASE 额外检查法案、registry 与 clean Git commit。该检查不写文件、不输出 PASS artifact；正式裁决统一由 `quality:check` 完成。

47. 读取规则

> READ_ONLY：读取回答问题所需的权威和目标文件，不落盘；
> CHANGE：读取相关法案、registry 和目标文件；
> READY：在 CHANGE 基础上读取全部 active law、Manifest、blueprint、spec 与验收合同；
> RELEASE：读取完整候选、全部 active laws、registry 和发布证据。

说明书版本、路径不符合本规范或 AGENTS 违反第 45 节时，CHANGE、READY、RELEASE 停止；READ_ONLY 仍可读取并报告事实。

第八部分 | 质量与证据

48. Static Audit 的边界

Static Audit 只证明当前候选的文件、文本、AST、类型、schema、合同、依赖、配置和迁移声明。

Static Audit 不得启动外部系统，不得将测试夹具、历史报告或截图认定为当前运行证据，不得输出 RELEASE verdict。

49. 原生 Git 候选身份

CHANGE 以当前 Git HEAD 到工作树的实际差异为候选分母，可以在未提交状态下迭代。

READY 与 RELEASE 只接受整个仓库 `git status --porcelain` 为空、没有 untracked 或 unmerged 项的 Git commit；不得以“与本次任务无关”为由忽略脏文件。候选身份精确等于该仓库的原生 Git commit；法案、registry、代码、测试和仓库内合同已经包含在该 commit 的 tree 中，不再生成自定义候选散列身份或第二份文件 hash 清单。

跨仓 RELEASE 不把多个 commit 再计算成联合 hash，只在唯一质量报告中列出每个必要参与仓的 `repo + gitCommit + contractId + version`。

50. candidate drift

质量检查开始后，候选发生变化时：

> 当前检查失败；
>
> 本次临时结果作废；
>
> 不得发布质量报告；
>
> READY 或 RELEASE 必须重新读取当前 Git commit。

CHANGE 不生成 candidate receipt。READY 与 RELEASE 报告都绑定原生 Git commit；RELEASE 另外绑定必要参与仓的原生 Git commit 和外部证据 receipt。

51. 一套普通治理测试

治理测试可以按 domain 或技术类型拆分文件，由仓库现有测试框架组织。只有 canonical quality 入口可以汇总治理结论；测试文件数量和目录结构不构成第二质量入口。

测试至少覆盖：

```
法案 parser;
lawId 与当前版本关系;
active law source allowlist;
Manifest Owner;
registry deterministic bytes;
scope;
证据分层;
禁止无事实依据的 PASS。
```

测试、检查器和辅助代码可以共享，一次执行可以返回多个 lawId 的结果。法条分母只来自两份 active law、Manifest 与唯一 registry；共享执行必须拆分成逐 lawId 结果，且不得以一个 domain 汇总 PASS 代替其中任一法条的裁决。

52. 静态证据分母

静态证据分母精确为：当前候选中命中 active law `appliesTo` 的路径，减去命中 `excludes` 的路径，再加入该 lawId 全部 active contractRefs。CHANGE 的当前候选是工作树实际差异，READY 与 RELEASE 的当前候选是 clean Git commit。每个路径必须命中该 domain Manifest 的 `ownedPaths` 或 `observedPaths`，否则 FAIL。

quality 按同一确定性算法直接计算该分母；共享检查器只能消费这组路径，不得换算法、缩小范围或按结果裁剪。纯 absence 检查允许分母为空；进入 READY 或 RELEASE 报告时，仍必须在该 lawId 的证据中记录 `scope:<lawId>#appliesTo-minus-excludes`。报告的 Git commit 已绑定查询输入与仓库内目标文件，不再封存范围集合或法条范围的重复 hash。

discovery、read、decode 或 parse 失败必须 FAIL。

以下路径不得作为业务实现已经成立的充分证据：

```
tests/**
fixtures/**
examples/**
artifacts/**
coverage/**
docs/history/**
scripts/audit/reports/**
```

53. 共享检查器合同

检查器可以覆盖一个或多个 lawId，也可以共享模块、解析器和测试构造器。quality 必须执行仓库唯一现役检查集合，并以全部返回结果的 lawId 与当前 required lawId 分母精确相等来发现漏执行、重复执行和额外执行；内部注册和模块组织由仓库决定，不进入 Manifest 或 registry。检查器只能读取 quality 按第 52 节计算的当前候选分母，并返回

第 54 节逐 lawId 结果。禁止固定 PASS、空证明、隐匿或忽略异常、直接写最终报告，或用 domain 汇总结果代替逐法结果。

54. 逐 lawId 裁决

Static、Exam、Dynamic 统一使用同一个轻量逐法结果合同。共享检查器或执行器对每个实际处理的 lawId 精确返回：

```
{
  "lawId": "LAW-...",
  "assertions": [
    {
      "clauses": ["summary", "forbidden:0"],
      "evidenceMode": "static.ast",
      "passed": true,
      "refs": ["ast:<repo-relative-path>#<selector>"]
    }
  ],
  "findings": []
}
```

`clauses[]` item 只允许 `summary` 或 `forbidden:<zero-based-index>`，直接指向 law-record 的规范条款，不建立全局检查身份。本等级每个 required evidenceMode 必须至少有一条 assertion；全部 required evidenceModes 的 assertions 合并后，clauses 并集必须精确覆盖该 lawId 的 `summary` 和每个 `forbidden[]` 项。一个条款可以由一个或多个与实际证明对象相符的 evidenceMode 裁决，但不得为了凑覆盖而复制到不适用的 evidenceMode。同一 assertion 可以覆盖由同一次检查共同裁决的多个条款。每条 assertion 只允许 `clauses/evidenceMode/passed/refs`，clauses 与 refs 均须非空、排序且无重复；同一 evidenceMode 下完全相同的 clauses 组合不得重复。纯 absence 检查必须引用第 52 节 scope。

Static、Exam、Dynamic 可以分别返回只属于自身类别的 assertions；quality 先按 `lawId + evidenceMode` 校验必需模式、重复项和证据类型，再按 lawId 合并执行上述条款并集覆盖检查。逐法结果顶层只允许 `lawId/assertions/findings`；finding 精确使用第 56 节的 `code/evidenceRefs` 结构。检查器不返回 PASS/FAIL；quality 根据 assertions 和 findings 统一裁决，并把 passed assertions 的 refs 按 evidenceMode 合并为第 56 节报告 evidence。任一 assertion 为 false 时必须产生 finding；任一 finding 也必须使对应 law FAIL。异常、规范条款漏检、缺失或额外 evidenceMode、重复 assertion、空 refs 均使该 law FAIL。这样保留逐条款执法；检查器按共享实现组织，逐法结果只使用本节轻量合同。

READY 与 RELEASE 对每个 active lawId 产生一个最终裁决。CHANGE 不生成全量逐 lawId 裁决。

law 裁决只允许：

```
PASS：本等级要求的证据完整且无 finding；
FAIL：要求不成立、禁止条件出现或本等级 required evidence 失败。
```

每个 law 同时列出 declared、required 与 deferred evidenceModes。READY 只允许 dynamic.* 进入 deferred；RELEASE 的 deferred 必须为空。

55. evidenceRef

仓库内 evidenceRef 精确使用 `<kind>:<repo-relative-path>#<selector>`；`scope` 使用 `scope:<lawId>#appliesTo-minus-excludes`。kind 只允许 `file/text/ast/json/yaml/sql/dependency/config/test/exam/dynamic/scope`。selector 必须稳定，locator 或 selector 中的 `#` 与 `:` 必须 percent-encode。当前报告的 Git commit 统一绑定仓库内字节，因此不得再追加逐文件 SHA-256。

仓库外或二进制证据使用 `<kind>:<real-locator>#<selector>@<native-receipt-or-sha256>`。只有真实浏览器、服务、数据库、存储、provider、授权、恢复、性能观察或未进入 Git 的二进制 artifact 可以使用该形式；receipt 必须可回查，不能用自然语言标签代替。

自然语言“见代码”“已经测试”“应该没问题”不是 evidenceRef。

56. 唯一逐法条质量报告

READY 与 RELEASE 每次只生成一份报告：

```
{
  "schemaVersion": 4,
  "level": "READY",
  "gitCommit": "<git object id>",
  "laws": [
    {
      "lawId": "LAW-...",
      "declaredEvidenceModes": ["static.ast", "dynamic.service"],
      "requiredEvidenceModes": ["static.ast"],
      "deferredEvidenceModes": ["dynamic.service"],
      "evidence": [
        {
          "evidenceMode": "static.ast",
          "refs": ["ast:<repo-relative-path>#<selector>"]
        }
      ],
      "outcome": "PASS",
      "findings": []
    }
  ],
  "participants": [],
```

```
  "outcome": "PASS"
}
```

报告顶层只允许 `schemaVersion/level/gitCommit/laws/participants/outcome`；law 对象只允许 `lawId/declaredEvidenceModes/requiredEvidenceModes/deferredEvidenceModes/evidence/outcome/findings`。

`level` 只允许 READY、RELEASE。两级的 `gitCommit` 均必须等于当前 clean Git commit；RELEASE 的 deferredEvidenceModes 必须为空。

`laws[]` 按 lawId 排序且精确覆盖全部 active lawId。declared 精确等于 law.evidenceModes，required 与 deferred 互斥且并集等于 declared。PASS law 的 evidence 按 evidenceMode 排序并与 requiredEvidenceModes 精确相等，每个 required mode 的 refs 非空；FAIL law 的 evidence 只保留已取得非空正向 refs 的 required mode，缺失或失败的证明写入 findings，不得填空 refs、额外 mode 或虚构证据。任一 lawId 或 evidenceMode 缺失、重复、额外、空证据、检查异常或 finding 均使 law FAIL。顶层只在全部 law PASS 时 PASS。

evidence item 只允许 `evidenceMode/refs`。仓库内 refs 由 `gitCommit` 绑定；仓库外或二进制 refs 必须依第 55 节携带真实 locator 和原生 receipt 或 SHA-256。

finding 对象只允许 `code/evidenceRefs`；code 必须是确定性阻断码，evidenceRefs 必须非空。PASS 对象的 findings 必须为空，FAIL 对象的 findings 必须非空。

`participants[]` 在 READY 必须为空；RELEASE 只登记必要跨仓参与者，每项精确为 `{repo,gitCommit,contracts}`，`contracts[]` 每项精确为 `{contractId,version}`。必要 participant 精确等于当前仓全部 active `cross-repo-dependency` 所引用的 providerRepo；provider 自身发布合同不因存在 consumer 而反向要求 consumer 同步参与。quality 必须从 participant 的该 Git commit 中扫描 active `cross-repo-contract` specs，并按 `providerRepo + contractId + version` 找到唯一匹配项；零项、多项、版本不兼容、participant commit 缺失或漂移均使 RELEASE FAIL。数组排序、无重复；不得基于 participants 另造联合候选身份。

顶层与 law outcome 只允许 PASS、FAIL。不存在 `NOT_APPLICABLE`、`SKIPPED_PASS`、warning PASS 或 partial PASS。

报告不得再拆成平行 decision、Owner receipt、CI receipt 或其它同义 verdict 层。

57. 报告写入

唯一 writer 只写第 56 节报告：

```
本地 READY、RELEASE：scripts/audit/reports/law-quality.report.json；
CI：仓库外 ephemeral path。
```

报告只证明对应 `gitCommit/level` 以及 RELEASE 时列明的 participants，不取得法案主权，不作为下一次证据输入。

58. 分级质量流程

```
CHANGE：actual diff -> 受影响 project checks 与普通测试 -> PASS | FAIL；
READY：clean Git commit -> source -> registry -> ownership -> 全部 projectChecks -> 全部 active Static -> 全部 active Exam -> 治理核心反例测试 -> 一份逐法条报告 -> PASS | FAIL；
RELEASE：同一 clean Git commit -> 全部 projectChecks -> 全部 active evidenceModes -> required participants -> commit recheck -> 一份逐法条报告 -> PASS | FAIL。
```

不得在各步骤之间生成可独立宣称整体通过的中间 PASS 文件。

59. 三类证据不得相互替代

```
Static Audit 证明静态结构；
Exam 证明确定性行为；
Dynamic Proof 证明真实外部运行。
```

任何证据只能证明其实际观察到的对象。

60. Exam 的建立与适用

只有 active law 声明 `exam.*` evidenceMode 时才建立 Exam；law 未声明时不得追加。READY 与 RELEASE 执行全部 active Exam。

Exam 代码可以共享，一次运行可以覆盖多个 lawId，但必须在本次 quality run 中按第 54 节产生逐 lawId assertions，不得复用历史结果。Exam assertion 只能使用该 law 声明的 `exam.*`，refs 必须定位命令或用例、输入或 seed、超时和输出；这些信息必须足以在同一 Git commit 上复放。Exam 不返回最终 PASS，finding 和 failed assertion 统一由 quality 裁决。

61. Dynamic Proof 的时点

只有 active law 声明 `dynamic.*` evidenceMode 时才建立 Dynamic Proof。RELEASE 执行全部 active Dynamic Proof；CHANGE 与 READY 不执行，READY 必须如实列入 deferredEvidenceModes。

Dynamic Proof 只证明真实浏览器、服务、数据库、存储、provider、授权、恢复或性能事实。它必须在本次 RELEASE 的 Git commit 确定后直接观察并按第 54 节产生逐 lawId assertions，不得复用历史观察或旧 artifact 作为当前结果。Dynamic assertion 只能使用该 law 声明的 `dynamic.*`，refs 必须按第 55 节记录真实 locator、观察值与原生 receipt 或 SHA-256。Dynamic Proof 不返回最终 PASS，finding 和 failed assertion 统一由 quality 裁决；凭证不得写入仓库或报告。

执行不可逆动作前，必须取得授权人对该具体动作的明确批准。cleanup 失败即 Dynamic Proof 失败。

外部动态条件暂时不可用时：

```
CHANGE 可以完成；
READY 可以按其静态和测试条件裁决；
RELEASE 必须失败或等待。
```

62. 证据适用性

`appliesTo/excludes`、Manifest paths 和 contractRefs 只定义检查分母，不得使 active law 在 READY 或 RELEASE 退出分母。

```
CHANGE：只执行与实际差异相交的检查，不产生全局裁决；
READY：全部 active lawId 进入报告，全部 static.* 和 exam.* 为 required，全部 dynamic.* 为 deferred；
RELEASE：全部 active lawId 的全部 declared evidenceModes 为 required，deferred 为空。
```

分母必须同时满足：

```
activeLawIds
= 全部 Manifest.lawIds 的并集
= registry.laws lawIds
= report.laws lawIds

顶层 PASS 时的 required (lawId, evidenceMode)
= report.laws 中具有非空 refs 的 evidence (lawId, evidenceMode)
```

上述等式均为集合精确相等；缺失、重复、额外项或临时降低 evidenceModes 均 FAIL。

63. 唯一质量入口

仓库只保留一个 canonical quality 入口 `quality:check`，并接受 CHANGE、READY、RELEASE 三级参数。编程语言和内部文件组织由仓库决定。

level 必须与第 35 节分类一致。

63.1 CHANGE

只根据实际 Git 差异执行可确定的 source 授权门、相关 project checks 和普通测试，不读取或推断授权人原始请求，不自动证明范围一致性或用户原有改动保护。CHANGE 不执行全仓 law history、全量 registry/ownership、全量治理测试或逐 lawId 报告。机器 PASS 只表示这些已执行检查通过；执行者完成第 44 节任务上下文核对后，才可报告本次 CHANGE 闭合。

63.2 READY

对 clean Git commit 执行 source、registry、ownership、全部 projectChecks、全部 active Static、全部 active Exam、治理核心反例测试和逐 lawId 报告。PASS 只表示候选达到 READY 条件。

63.3 RELEASE

对同一 clean Git commit 重新执行全部 projectChecks、全部 active evidenceModes 和必要跨仓 participant 核验，并把结果写入同一份逐法报告。全部通过后才可产生 `SYSTEM_PROVEN`。

64. CI 与 RELEASE 来源记录

64.1 CI

CI 只能调用同一 canonical quality 入口。普通 PR 使用 CHANGE 或 READY；自动事件不得提升为 RELEASE。

CI 报告写入仓库外临时目录，不修改 source、registry 或仓库内 report cache。最终成功或失败只认质量进程退出码。

64.2 RELEASE 来源记录

RELEASE 不生成第二发布证明包或第二 receipt。当前仓 Git commit、按第 56 节从 active consumer dependencies 推导的必要 provider participant、合同版本，以及仓库外 Dynamic Proof 的真实 locator 与 receipt，全部直接进入唯一质量报告。provider 可以是此前已经发布的 clean commit，不要求与 consumer 在同一任务修改；质量结束前再次读取所有 participant commit，任一漂移即 FAIL。

第九部分 | 根治理规则与建制

65. blank bootstrap

空白仓首次建制只建立：

> 本说明书；
>
> AGENTS；
>
> 基础法案与功能法案；
>
> 满足第 66 节准入条件的当前项目根治理法条；不存在此类风险时不预建元法案；
>
> 按实际生产路径建立的 Manifests；
>
> 唯一 law registry；
>
> 一套普通治理测试；
>
> READY 与 RELEASE 共用的唯一逐法条报告 writer 与目标路径；bootstrap 不预写报告；
>
> 第 68 节三个公共治理命令。

blueprint、spec、迁移桥和跨仓合同只在第 22、24、43、69 节条件成立时建立；Exam 与 Dynamic Proof 只依 active law `evidenceModes` 建立。不得预建无实质内容的占位对象。

bootstrap 完成 CHANGE 与 READY 后可以继续分步建设；只有进入 RELEASE 时才要求全部 RELEASE required evidence 与必要跨仓参与仓 commit。

66. 项目根治理规则

基础法案只保存项目确实需要长期维持、能够直接约束生产或交付行为的根治理法条。本说明书不规定任何必备 lawId、lawKey、编号、capabilityDomain 或法条数量；当前根治理分母只来自两份 active law source，并随获得授权的新增、合并、修订和退役自然变化。

根治理法条只有同时满足以下条件才可进入基础法案：

> 存在一个尚未被本说明书或现役 active law 完整覆盖的长期项目风险；
>
> 该风险具有明确 Owner、适用范围、禁止面和与风险事实相符的 evidenceModes；
>
> 该规则直接约束生产、交付、权限、主权或跨版本行为，而不是只约束治理工具的内部实现；
>
> 该规则能够按第 14.3 节形成一个可独立修订或退役的规范主题；
>
> 普通测试、类型、schema 或本说明书的直接检查不足以替代该规则。

以下内容直接由本说明书和治理核心测试约束，不得仅为证明法案系统自身而复制成固定 `law-system`、`audit-system` 或其它元法案名单：

```
active law source 数量与 parser 语法；
Manifest、registry 和报告 schema；
compiler、projection check、quality 与 report writer 的唯一性；
逐 lawId、逐规范条款和逐 evidenceMode 的完整性；
固定 PASS、空证据、隐匿或忽略异常、错误证据类型和 candidate drift 的失败关闭；
治理测试、fixture、目录、命令和内部模块组织。
```

新增根治理法条按第 71 节办理；语义重叠时必须优先修订或合并现役法条，不得仅因出现新实现组件而新增 lawId。退役按第 72.2 节办理；只要剩余 active law 与本说明书仍完整覆盖实际风险，任何旧 lawKey 都可以删除，不得设置永久最小数量。

quality 只以当前 active lawId 集合执法，不维护“必须存在的历史 lawKey”清单。独立审查检查实际风险是否存在无主、无规范或无证据的空洞，但不得以固定名称、固定数量或旧版本名单代替该判断。

67. 禁止递归元法案工厂

治理系统不得固定要求：

```
缺少独立长期项目风险、只为证明治理工具自身而建立的元法案；
每条法案一份 spec 文件；
每条法案一份 blueprint 文件；
每条法案一份 predicate 文件；
每条法案一份 Exam 文件；
每条法案一份反例文件；
多份内容重叠的 generated projection；
多层 decision、receipt 和 report verdict。
```

禁止的是机械复制规范、文件和执行身份。新增治理 law 只有在满足第 66 节全部准入条件时才合法；共享检查器仍必须按第 54、56 节返回逐 lawId 裁决。

业务 law 按 capabilityDomain 写入功能法案；其 blueprint、spec、Exam、Dynamic Proof 和专项测试均按实际拓扑、合同及 `evidenceModes` 建立。

68. 唯一命令面

跨实现稳定的公共治理命令身份只允许固定为：

```
audit:build
audit:projection:check
quality:check
```

它们分别承担 registry 构建、registry 复验和唯一质量裁决。变更前的 Git 状态查看使用仓库原生命令，普通治理测试由 `quality:check` 调用，不冻结独立命令名。编程语言、内部脚本文件名、模块拆分、依赖和测试框架均由仓库决定，不属于本说明书冻结内容。

Exam 或 Dynamic Proof 存在时可以拥有由 `quality:check` 调用的内部执行命令；这些命令不得作为公共治理入口，不得写最终报告或形成第二质量 verdict。

第十部分 | 既有仓库迁移与新增能力

69. 既有仓库迁移

既有仓库允许分步迁移。存在迁移桥时，必须在第 24 节 `specKind=migration` 的 spec 的 `contract` 中记录：

```
{
  "currentAuthority": "<identity>",
  "targetAuthority": "<identity>",
  "compatibilityMechanisms": [],
  "startDate": "YYYY-MM-DD",
  "monitoring": [],
  "rollbackCondition": "...",
  "maximumDurationDays": 1,
  "removalDate": "YYYY-MM-DD",
  "removalAcceptance": []
}
```

没有迁移桥的普通分步建设不创建 migration spec 或其它迁移 artifact。

`maximumDurationDays` 必须按实际迁移期限填写正整数；上例的 `1` 只表示字段类型，不是默认值或全系统上限。`removalDate` 不得晚于 `startDate` 加 `maximumDurationDays`。

规则固定：

1. 中间步骤只能声明实际完成的 CHANGE 或 READY；最终步骤通过全部发布条件后才可声明 RELEASE。
2. 旧、新治理并存时必须指定当前裁决主权，不得双主权。
3. 临时 bridge、adapter 或 dual-read/write 依第 43.2 节登记和删除。
4. 完成最终 RELEASE 前，旧 normative source、旧 registry、旧 quality verdict 和过期 bridge 必须清零。
5. 治理迁移不得顺带修改未授权业务功能或工具链。

70. 新增 capabilityDomain

新增 domain 必须取得授权人的明确批准，并在首次生产写入前确定：

```
domain 名称;
primary Owner;
active lawIds;
ownedPaths;
```

> 与相邻 domain 的合同边界；
>
> 本阶段交付等级。

施工图、spec 依实际拓扑和合同建立，Exam、Dynamic Proof 只依 active law 的 `evidenceModes` 建立。不得因尚未进入 RELEASE 而创建无实质内容的占位文件，也不得把未完成项写成已完成。

71. 新增 lawId

新增 lawId 必须同时完成：

> 唯一 law-record；
>
> 合法 lawId、domain 和当前版本关系；
>
> Manifest.lawIds；
>
> 确认现有共享检查已覆盖新法条的 summary、全部 forbidden 与全部 evidenceModes；不能覆盖时修改相应检查代码和普通测试；
>
> 唯一 registry 重建；
>
> 进入 READY 或 RELEASE 时的逐 lawId 质量报告。

只有法条实际涉及机器合同才增加 spec；只有其 `evidenceModes` 声明对应类型时才增加 Exam 或 Dynamic Proof。不得机械创建一法一文件，也不得省略逐 lawId 的最终裁决。

72. lawId 修订和退役

72.1 修订

summary、forbidden、appliesTo/excludes、Owner、evidenceModes、tier 或依赖义务发生规范变化时，创建下一 major version并记录 supersedesLawId。同一 law-amendment 必须删除旧 active block，同步更新 Manifest.lawIds、必要的共享检查和普通测试，并重建 registry；新、旧 major 不得同时 active。

法条 major 与 spec version 独立：只改变业务合同而未改变法条义务时，只递增 spec version，不触发本说明书或无关 lawId 修订。破坏性合同变化可以在声明的兼容窗口内保留旧、新 spec version；必须满足第 43.2 节，并在期限内删除旧版本。兼容窗口不改变唯一规范主权：新版本生效范围和旧版本退休范围必须明确。

72.2 退役

退役必须删除 active block 和 Manifest 当前引用，移除只为该法条存在的检查分支，并通过唯一 builder 重建 registry 以移除当前记录；不得手改 generated registry。历史只保留在 Git 或明确 HISTORY_ONLY 路径。

被其它 active law、合同或消费者继续引用时不得宣称退役完成。

第十一部分 | 必须阻断的漂移

73. 架构漂移

违反第 4、18、25、26 或 43.2 节的权威、Owner、唯一主链或迁移期限，即构成架构漂移并必须 FAIL；不得在本节为同一事实建立第二套禁令。

74. 范围漂移

违反第 35 至 44 节的任务等级、授权范围、用户改动保护或实际差异核对，即构成范围漂移并必须停止；机器检查不得代替第 44 节执行者核对。

75. 证据漂移

违反第 48 至 62 节的候选、证据类型、规范条款覆盖、分母或报告要求，即构成证据漂移并必须 FAIL；文件存在、数量存在、fixture、历史结果或较低交付等级均不得替代规定证据。

76. 迁移漂移

迁移桥、用于迁移的 feature flag、dual-read、dual-write、shadow traffic、adapter 和兼容窗口依第 43.2、69 节裁决；缺少任一必备字段、旧新主权不明、到期未删除或将迁移状态认定为 RELEASE，均构成迁移漂移并必须 FAIL。

77. 文档漂移

违反第 8、13、15、23 至 25、45 或 57 节的文档主权和内容边界，即构成文档漂移并必须 FAIL；执行者解释、施工记录、历史材料和投影对象不得产生新规范。

第十二部分 | 测试与独立自检

78. 普通测试与治理核心反例

仓库保留一套普通治理测试，不建立逐法反例身份、反例目录或“一条法案一个反例”的登记系统。

治理核心反例至少覆盖：

> 漏掉、重复或额外返回一个 active lawId；
> 漏掉、重复或额外返回一个 required evidenceMode；
> 漏检 summary 或任一 forbidden 条款；
> 同一 evidenceMode 下重复完全相同的 clauses assertion；
> 固定 PASS；
> 空 evidence refs；
> finding 未使 law 和顶层失败；
> 检查器异常被隐匿或忽略；
> Static、Exam、Dynamic 证据类型相互替代；
> registry 陈旧；
> candidate drift；
> 跨仓 provider 合同缺失或版本不兼容；
> RELEASE 缺 required Dynamic Proof 或 participant commit。

这些反例可以共享 fixture、构造器和测试文件；一个用例可以同时证明多个 lawId 会失败，不要求唯一 ID 或写入质量报告。业务规则需要负向行为验证时，直接写普通领域测试，不升级为治理 metadata。

CHANGE 只执行与实际差异相交的测试。READY 执行全部治理核心反例和 active Static/Exam 相关普通测试。RELEASE 在 READY 基础上验证 Dynamic Proof 缺失、错误类型和跨仓不兼容必定失败；不得对真实外部系统执行破坏性故障注入。

79. 独立自检

说明书定稿、治理主链变化或大规模迁移后，独立审查至少检查：

> 权威是否唯一；
> 任务范围是否越界；
> registry 是否忠实投影 active laws；
> 逐法 required evidenceModes 和报告分母是否精确相等；
> 逐法条报告是否使用正确证据；
> 每个 PASS law 的 required evidence 是否非空且可复放；

> 治理核心反例是否能阻止漏法、空证据、隐匿异常和无事实依据的 PASS；
>
> CHANGE、READY、RELEASE 是否被如实区分；
>
> 迁移机制是否具有期限并可删除；
>
> 是否存在固定 PASS、遗漏分母或第二 verdict。

审查输出 finding，不以总体评价或覆盖率代替具体证据。

第十三部分 | 最终验收与完成定义

80. 法案正文验收

法案正文只按第 8 至 16、66、71、72 节验收。任一 source、front matter、law-record、版本关系、规范主题、证据等级或根治理准入条件不成立即 FAIL；不得另设固定根 lawKey 或数量清单。

81. registry、施工图与规格验收

registry、blueprint 和 spec 只按第 20、22 至 34、56 节验收。唯一性仅指治理 registry projection；业务生成物不受该限制。任何分母不等、陈旧 projection、无必要性的 blueprint/spec、空模板或跨仓合同不匹配均必须 FAIL。

82. Owner 与变更验收

Owner 与变更只按第 17 至 21、35 至 47 节验收。执行者必须单独完成第 44 节任务上下文核对；机器结果不得被认定为已经读取授权人原始请求或识别修改前用户改动。

83. 交付与迁移验收

交付与迁移只按第 3、35、40、43、58、61 至 64、69 节验收。较低等级、阶段成果或迁移状态不得认定为较高等级；外部 Dynamic Proof 不可用只阻断 RELEASE，不阻断符合本说明书的 CHANGE。

84. 质量验收

质量只按第 48 至 64、78、79 节验收。lawId、规范条款或 evidenceMode 分母不等，证据为空或类型错误，异常被隐匿，finding 未失败关闭，candidate drift，第二 verdict，或治理核心反例未阻断相应错误时，均必须 FAIL。

85. 状态与完成定义

CHANGE_PASS：与当前实际差异相交的机器检查通过，且执行者已经在当前任务上下文中完成授权人请求与用户原有改动核对；该状态不声称仓库命令读取过授权人原始请求；

READY：clean Git commit 的全部 projectChecks 通过，全部 active lawId 进入报告，全部 Static、Exam 与治理核心反例通过，仅 `dynamic.*` deferred；

SYSTEM_PROVEN：当前 RELEASE Git commit 的全部 projectChecks、active evidenceModes 与 required 跨仓参与者通过。

附录 A | AGENTS.md 固定正文

bootstrap 生成的 AGENTS 规范性正文精确为：

```
# Codex Governance Entry

本仓库使用冻结的法案系统。

1. READ_ONLY 直接读取并回答，不写入任何文件。
2. CHANGE 开始修改前只读查看目标路径的 Git 状态，不得覆盖、删除、恢复或移动未获授权的用户改动。
3. CHANGE 的实际差异必须与授权人的请求及完成该请求不可分割的直接关联对象一致。
4. READY 不等于 RELEASE；静态检查、普通测试或部分证据不得被认定为已经完成生产发布。
5. READY、RELEASE 使用原生 clean Git commit；RELEASE 执行全部 active law 声明的 Dynamic Proof。
6. 只允许一份 active law corpus、一份 law registry；每个生产路径和合同只允许一个 primary Owner；只允许一个 quality verdict。
7. 迁移桥、用于迁移的 feature flag、dual-read/write 或 shadow traffic 必须有 Owner、监控、期限、删除日期和删除验收。
8. 本说明书与 active law 只在授权人分别明确批准对应修改后变更。
9. 任一 finding、候选漂移、错误证据类型或无事实依据的 PASS 必须失败关闭。
```

`AGENTS.md` 中上述正文由本说明书授权，但不属于 active law source，也不由旁路冻结文件绑定；其符合性由治理入口检查。

附录 B | 最小机器合同

B.1 Manifest

精确使用第 20 节；本附录不维护缩略 schema。

B.2 按需 blueprint machine block

精确使用第 22 节；本附录不维护第二份 machine block。

B.3 按需 spec

公共外壳精确使用第 24 节；B.6、B.7 分别定义两类跨仓 contract payload。

B.4 law registry

精确使用第 27、28 节；本附录不维护第二份 registry schema。

B.5 逐法条质量报告

精确使用第 56、57 节；本附录不维护第二份报告 schema。

B.6 跨仓 provider contract payload

本节不是独立 artifact；它是第 B.3 节 spec 在 `specKind=cross-repo-contract` 时的 `contract` 内容：

```
{
  "contractId": "provider-domain.contract",
  "version": 1,
  "providerRepo": "owner/repo",
  "providerDomain": "provider-domain",
  "authorityArtifact": {
    "path": "repo/relative/path",
    "selector": "stable-selector"
```

```
  }
}
```

B.7 跨仓 consumer dependency payload

本节不是 provider 合同副本；它是第 B.3 节 spec 在 `specKind=cross-repo-dependency` 时的最小依赖声明：

```
{
  "contractId": "provider-domain.contract",
  "version": 1,
  "providerRepo": "owner/repo"
}
```

B.6、B.7 对象均拒绝未知字段。consumer dependency 不得复制 authorityArtifact、provider contract 字段、provider commit 或任何合同 hash。provider 与 consumer 可以独立部署；合同内容变化递增 `version`，破坏性改变同时声明兼容窗口和迁移期限。

READY 只验证 consumer dependency 的 schema、唯一性和本仓使用，不读取未绑定的 provider 工作树，也不宣称跨仓兼容已经动态成立。RELEASE 按第 56 节从必要 provider participant 的原生 Git commit 中扫描唯一 active provider contract，并核对 `contractId + version + providerRepo`；repo、Git commit 和合同版本写入同一份质量报告，不建立额外证明包。